



T.C
DOKUZ EYLÜL ÜNİVERSİTESİ
FEN FAKÜLTESİ
BİLGİSAYAR BİLİMLERİ BÖLÜMÜ

BİL 2004 VERİTABANI ANALİZİ
2025-2026 BAHAR DÖNEMİ PROJESİ
DOLANDIRICILIK TESPİT SİSTEMİ

Berke BOZCA – 2024280014
Nazrin MAMMADOVA – 2023280043
Buse OZAN – 2024280046

Öğretim Üyesi: Prof. Dr. Emel Kuruoğlu Kandemir

ÖNSÖZ.....	2
1. GİRİŞ	4
1.1. Projenin Amacı ve Kapsamı.....	4
1.2 Projenin Tanımı ve Dolandırıcılık Tespit Gereksinimleri	4
2. VERİTABANI TASARIMI.....	5
2.1. Varlıklar ve Öznitelikler.....	5
2.1.1. Kullanıcılar (Users).....	5
2.1.2. Roller (Roles).....	5
2.1.3. Hesaplar (Accounts)	5
2.1.4. İşlemler (Transactions).....	6
2.1.5. Giriş Takibi (Login Tracking)	6
2.1.6. Risk Analizi (Risk Analysis).....	6
2.2. Normalizasyon ve İlişkisel Tablo Yapısı	6
2.3. Varlık - İlişki (ER) Diyagramı.....	9
2.3.1 İlişkisel Bağlantılar ve Kardinalite	11
3. VERİTABANI UYGULAMASI.....	13
3.1. Veritabanı Tablolarını Oluşturma (DDL)	13
3.2. Trigger İşlemleri	15
3.3. Stored Procedure – Riskli İşlemlerin Listelenmesi.....	16
3.4. Function - Kullanıcının Toplam Gönderdiği Para	17
3.5. Örnek Veriler (INSERT).....	18
3.6. Update Örneği	19
3.7. Delete Örneği - Örnek İşlem Silme.....	19
3.8. Gelişmiş Sorgular.....	19
3.9. Procedure Çalıştırma	21
3.10. Function Çağırma	22
3.11. Tüm Tabloları Gösterme	22
4. UYGULAMA/ARAYÜZ ENTEGRASYONU	23
4.1. C# Arayüz Entegrasyonu ve Veri Seçimi Mantığı (Teknik Özet)	24
5. KAYNAKÇA.....	25

ÖNSÖZ

Bu çalışma, Dokuz Eylül Üniversitesi Bilgisayar Bilimleri Bölümü BİL 2004 Veritabanı Analizi dersi kapsamında hazırlanmış bir dönem projesidir. Proje kapsamında, finansal işlemler üzerinden şüpheli aktiviteleri tespit edebilen bir Dolandırıcılık Tespit Sistemi için veritabanı tasarımı ve uygulaması gerçekleştirilmiştir.

Çalışma sürecinde veritabanı tasarımı, normalizasyon ve SQL uygulamaları temel alınmış ve geliştirilen sistem C# arayüzü ile desteklenmiştir. Projeye ait kaynak kodlar ve ilgili dokümanlar GitHub platformu üzerinden paylaşılmıştır. (<https://github.com/Nezrin25/Dolandiricilik-Takip-Sistemi>).

Rapor hazırlanırken akademik dürüstlük ilkelerine bağlı kalınmış olup metinlerin düzenlenmesi ve ifade biçiminin geliştirilmesi amacıyla yapay zekâ araçlarından (ChatGPT ve Gemini) destek alınmıştır. Tüm içeriklerin sorumluluğu proje ekibine aittir.

1. GİRİŞ

1.1. Projenin Amacı ve Kapsamı

Bu projenin amacı, finansal işlemleri analiz ederek şüpheli aktiviteleri tespit edebilen uygun bir **Dolandırıcılık Tespit Sistemi (Fraud Detection System)** veritabanı tasarımı oluşturmaktır. Günümüz dünyasında dijital finansal işlemlerin artmasıyla birlikte dolandırıcılık faaliyetleri de önemli ölçüde artmış olup bu tür riskleri erken aşamada belirleyebilen sistemlere duyulan ihtiyaç giderek artmaktadır. Bu nedenle, kullanıcı hareketlerini takip eden ve riskli durumları belirleyebilen bir sistem üzerinde çalışılmıştır. Proje; kullanıcı yönetimi, hesap hareketleri, cihaz takibi ve risk analizini kapsayan bütünlük bir yapı sunar.

1.2 Projenin Tanımı ve Dolandırıcılık Tespit Gereksinimleri

Bir finansal teknoloji şirketi kullanıcılarının gerçekleştirdiği işlemleri analiz ederek şüpheli aktiviteleri tespit edebilen merkezi bir sistem geliştirmek istemektedir. Sistem, kullanıcıların hesap hareketlerini izleyerek belirli kurallara göre riskli işlemleri işaretlemeyi amaçlar.

- **Riskli İşlem Analizi:** Olağan dışı yüksek miktartlı transferler, çok kısa süre içerisinde tekrarlanan (spam) işlemler ve kullanıcının alışılmış konumlarından farklı bölgelerden yapılan girişler sistem tarafından öncelikli olarak mercek altına alınır. Bu kriterlere uyan her işlem otomatik olarak "şüpheli" olarak işaretlenir.

- **Kayıt ve Sınıflandırma:** Şüpheli bulunan her işlem, sistemin veri bütünlüğünü korumak adına ayrı bir tabloda arşivlenir. Bu kayıtlar, ilgili işlemle doğrudan ilişkilidir ve potansiyel tehlike durumuna göre düşük (low), orta (medium) veya yüksek (high) risk seviyeleriyle sınıflandırılır.

- **Cihaz ve Giriş Takibi:** Güvenlik katmanını güçlendirmek amacıyla, kullanıcıların sisteme giriş yaptığı cihaz bilgileri ve coğrafi konum verileri anlık olarak kaydedilir. Bu teknik veriler, dolandırıcılık analizinde doğruluğu artırmak için temel veri kaynağı olarak kullanılır.

2. VERİTABANI TASARIMI

2.1. Varlıklar ve Öznitelikler

Dolandırıcılık Tespit Sistemi veritabanı tasarımında yer alan temel varlıklar ve bu varlıkların özelliklerini belirleyen öznitelikler aşağıda detaylandırılmıştır. Bu yapı, finansal süreçlerin ve güvenlik analizlerinin veri tabanı seviyesinde eksiksiz takip edilmesini sağlar.

2.1.1. Kullanıcılar (Users)

Kullanici_No (PK): Her kullanıcıyı sistemde tekil olarak tanımlayan benzersiz numara.

Kullanici_Adi_Soyadi: Kullanıcının resmi kimlik bilgilerini tutar.

Rol_ID (FK): Kullanıcının sistemdeki yetki seviyesini (Admin/Customer) belirlemek için Roller tablosuna bağlanır.

2.1.2. Roller (Roles)

Rol_ID (PK): Roller tablosundaki her yetki türünü tanımlayan anahtardır.

Rol_Adi: Rolün ismini (Örn: "Admin", "Müşteri") saklar.

2.1.3. Hesaplar (Accounts)

Hesap_No (PK): Finansal işlemlerin yapıldığı her bir hesabın tekil numarasıdır.

Kullanici_No (FK): Hesabın hangi kullanıcıya ait olduğunu belirleyen bağlantıdır.

2.1.4. İşlemler (Transactions)

Islem_No (PK): Gerçekleştirilen her bir finansal hareketin benzersiz referans numarasıdır.

Gonderen_Hesap_No (FK): Paranın hangi hesaptan çıktığını belirler.

Alici_Hesap_No (FK): Paranın hangi hesaba aktarıldığını belirler.

Islem_Tipi: İşlemin türünü (Havale, EFT, Ödeme vb.) belirtir.

Islem_Miktari: Transfer edilen para miktarını tutar.

Islem_Tarihi: İşlemin yapıldığı anlık zaman damgasıdır.

2.1.5. Giriş Takibi (Login Tracking)

Giris_ID (PK): Her bir giriş denemesini kaydeden benzersiz ID'dir.

Kullanici_No (FK): Girişi yapan kullanıcının kimliğini doğrular.

Giris_Cihazı: Sisteme erişilen cihazın türünü (Mobil, Web vb.) saklar.

Giris_Konumu: Erişim sağlanan coğrafi lokasyonu (Şehir/IP) tutar.

Giris_Tarihi: Sisteme giriş yapılan zamanı kaydeder.

2.1.6. Risk Analizi (Risk Analysis)

Risk_ID (PK): Şüpheli işlem kayıtlarını tekil olarak tanımlar.

Islem_No (FK): Riskin hangi finansal işlemten kaynaklandığını belirtir.

Risk_Seviyesi: Analiz sonucunda atanan tehlike derecesini (Low, Medium, High) saklar.

2.2. Normalizasyon ve İlişkisel Tablo Yapısı

Veritabanı tasarımında veri tekrarını önlemek, veri bütünlüğünü sağlamak ve depolama verimliliğini artırmak amacıyla normalizasyon süreci uygulanmıştır. Tasarım, karmaşık bir yapıdan (1NF) başlayarak daha düzenli ve ilişkisel bir yapıya (3NF) dönüştürülmüştür. Her normalizasyon basamağında seçilen anahtarlar ve bağlantıları açıklanmıştır.

Birinci Normal Form (1NF):

1NF aşamasında, veritabanındaki tüm sütunların atomik (bölünemez) olması sağlanmış ve tekrarlanan gruplar ayıklanmıştır. Bu aşamada tüm veriler tek bir geniş tabloda toplanmıştır; ancak bu yapı veri tekrarına ve güncelleme anomalilerine (bir veriyi değiştirirken diğerlerini unutma riski) müsaittir.

- İşlemKayıtlar:(Kullanici_No(PK),İslem_No(PK),Kullanici_Adi_Soyadi,Kullanici_Rolu,Hesap_No,İslem_Tipi,İslem_Miktari,İslem_Tarihi,Giris_Cihazı,Giris_Konu,Risk_Seviyesi,Giris_Tarihi)

İkinci Normal Form (2NF):

2NF'e geçiş aşamasında, 1NF'deki tablodan kısmi bağımlılıklar kaldırılmıştır. Veriler, anahtar alanlara tam bağımlı olacak şekilde mantıksal gruplara ayrılarak farklı tablolar oluşturulmuştur.

1NF yapısında tüm veriler tek bir tabloda toplandığı için her satırı ayırt etmek adına ***Kullanici_No*** ve ***İslem_No*** birlikte kompozit anahtar olarak kullanılmıştır. Ancak bu durum, kullanıcı adının sadece kullanıcı numarasına bağlı olması gibi "kısmi bağımlılık" sorunları yaratır.

- Anahtar Seçimi: Kullanıcı bilgilerini, işlem bilgilerinden ayırmak için ***Kullanici_No*** kendi tablosunda PK olarak seçilmiştir.
- Bağlantı Mantığı: İşlemler tablosuna ***Kullanici_No (FK)*** eklenmiştir. Bu bağlantı, "bir işlemin mutlaka bir sahibi olmalıdır" kuralını teknik olarak zorunlu kılar.

Normalizasyonun ikinci normal forma uygun hali aşağıda belirtildiği gibidir:

- Kullanıcılar:(*Kullanici_No (PK)*,*Kullanici_Adi_Soyadi*,*Kullanici_Rolu*)
- İşlemler:(*Islem_No (PK)*,*Kullanici_No (FK)*,*Hesap_No*,*Islem_Tipi*,*Islem_Miktari*,*Islem_Tarihi*)
- Giriş Takibi:(*Giris_ID (PK)*,*Kullanici_No (FK)*,*Giris_Cihazı*,*Giris_Konumu*,*Giris_Tarihi*)
- Risk Analizi:(*Risk_ID (PK)*,*Islem_No (FK)*,*Risk_Seviyesi*)

Üçüncü Normal Form (3NF):

Son aşamada geçişli bağımlılıklar ortadan kaldırılmıştır. Örneğin; bir kullanıcının rolü kullanıcıya bağlıdır ancak rolün adı aslında rolün kendisine aittir. Bu tür ilişkiler ayrıştırılarak veritabanı en optimize hale getirilmiştir. Ayrıca "Hesaplar" tablosu oluşturularak bir kullanıcının birden fazla hesabı olabilmesi sağlanmış, "İşlemler" tablosu gönderici-alıcı ilişkisine göre güncellenmiştir.

- Anahtar Seçimi: Kullanıcılar tablosundaki "Rol Adı" bilgisi aslında kullanıcıya değil, rolün kendi tanımına aittir. Bu nedenle Rol Tablosundaki **Rol_ID** birincil anahtar (PK) olarak belirlenmiş ve roller merkezleştirilmiştir.

Bir kullanıcının birden fazla hesabı olabileceği gerçeğinden yola çıkarak hesaplar bağımsız bir varlık haline getirilmiş ve Hesaplar Tablosunda **Hesap_No** birincil anahtar (PK) olarak seçilmiştir.

Her bir risk analizi ve giriş hareketini tekil olarak takip edebilmek adına Giriş Takibi ve Risk Analizi tablolarında **Risk_ID** ve **Giris_ID** anahtarları PK olarak atanmıştır.

- Bağlantı Mantığı: Kullanıcı tablosu, Roller tablosuna bağlanarak her kullanıcının sadece sistemde tanımlı bir yetkiye sahip olması zorunlu kılınmıştır. Kullanıcı Tablosunda yer alan **Rol_ID** yabancı anahtar (FK) olarak seçilmiştir. Bu, "geçişli bağımlılığı" çözer.

Hesaplar tablosunda **Kullanici_no** yabancı anahtar (FK) olarak seçilerek her hesap bir kullanıcıya bağlanmıştır. Bu sayede hesapların kime ait olduğu veritabanı seviyesinde garanti altına alınır.

İşlemler tablosunda seçilen **Gonderen/Alici_Hesap_No** yabancı anahtarı (FK) sayesinde İşlemler artık kullanıcı numarası üzerinden değil, doğrudan **Hesap_No** üzerinden yürütülür. Bu bağlantı mantığı, bir işlemin kaynağını ve hedefini finansal kurallara uygun şekilde teknik olarak zorunlu kılar.

Risk Analizi tablosunda **Islem_No** yabancı anahtar (FK) olarak bağlanmıştır. Her risk kaydı bir yabancı anahtar ile ilgili işleme bağlanır; böylece "Hangi işlem için bu risk uyarısı verildi?" sorusunun cevabı veri bütünlüğü içerisinde korunur.

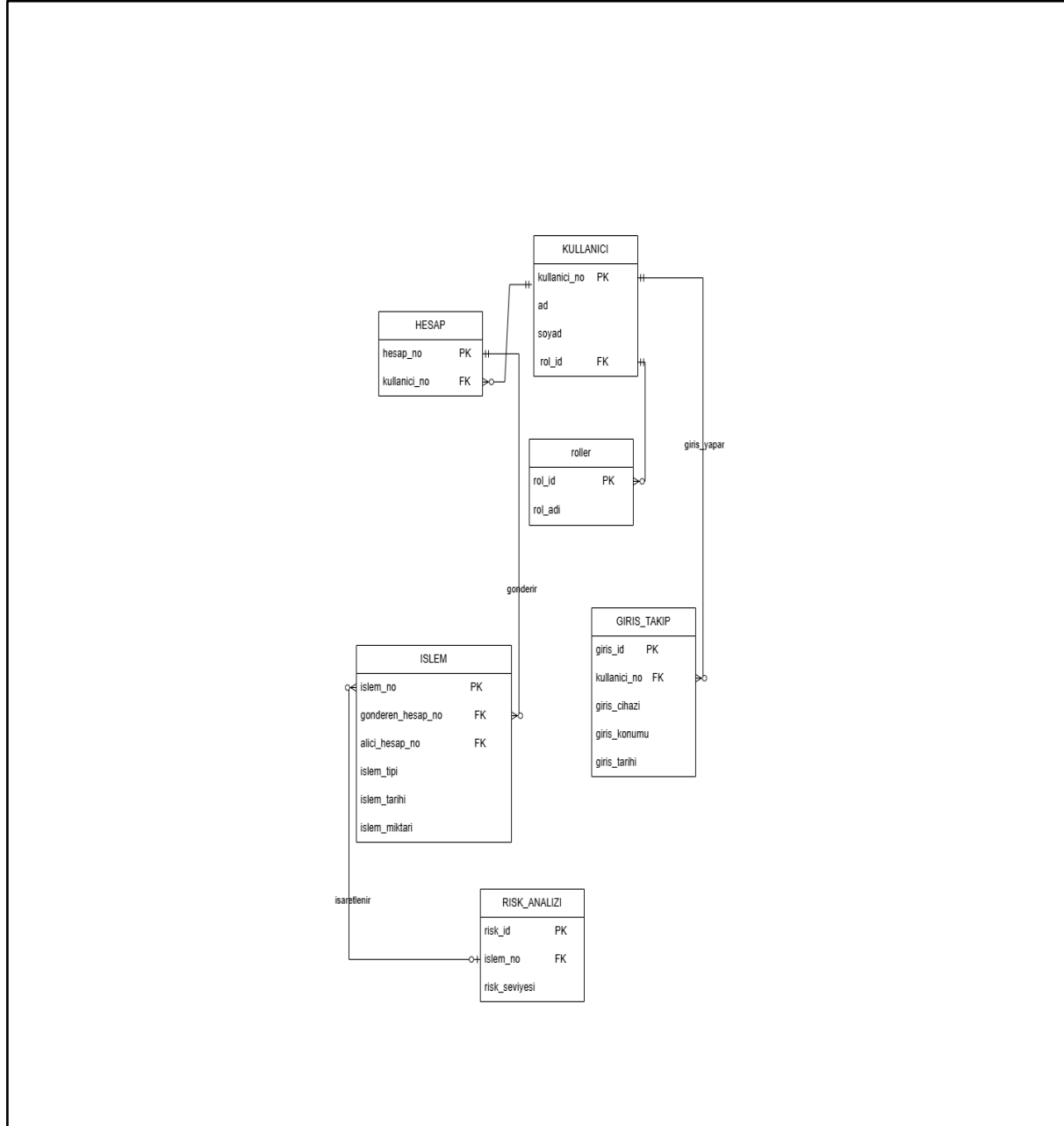
Normalizasyonun 3NF'e uygun hali aşağıda belirtildiği gibidir.

- Kullanıcılar: (*Kullanici_No* (PK), *Kullanici_Adi_Soyadi*, *Rol_ID* (FK))
- Roller: (*Rol_ID* (PK), *Rol_Adi*)
- Hesaplar: (*Hesap_No* (PK), *Kullanici_No* (FK))
- İşlemler: (*Islem_No*, (PK), *Gonderen_Hesap_No* (FK), *Alici_Hesap_No* (FK), *Islem_Tipi*, *Islem_Miktari*, *Islem_Tarihi*)
- Giriş Takibi: (*Giris_ID* (PK), *Kullanici_No* (FK), *Giris_Cihazı*, *Giris_Konumu*, *Giris_Tarihi*)
- Risk Analizi: (*Risk_ID* (PK), *Islem_No* (FK), *Risk_Seviyesi*)

2.3. Varlık - İlişki (ER) Diyagramı

Proje kapsamında tasarlanan **Dolandırıcılık Tespit Sistemi** veri tutarlılığını ve ilişkisel bütünlüğü sağlamak amacıyla normalize edilmiş altı temel varlık üzerine inşa edilmiştir.

Aşağıda, ER diyagramında yer alan varlıklar ve bu varlıklar arasındaki ilişkisel mantık teknik detaylarıyla açıklanmıştır.



Şekil 2.1. Veritabanı ilişkisel şeması

2.3.1 İlişkisel Bağlantılar ve Kardinalite

Sistemdeki varlıklar arasındaki bağlar, veri bütünlüğünü korumak ve "Dolandırıcılık Tespit Sistemi" senaryosunun gereksinimlerini karşılamak üzere belirli kardinalite (nicelik) kurallarıyla yapılandırılmıştır:

- **Kullanıcı - Rol İlişkisi (Many-to-One/N:1):**

Senaryo gereği her kullanıcı sistemde tek bir yetki grubuna (Admin veya Customer) dahil olabilir.

Diyagramda KULLANICI tablosundaki rol_id (FK), ROLLER tablosuna bağlanır. Bu yapı, binlerce kullanıcının tek bir rol tanımını paylaşmasına izin vererek veri tekrarını önler.

- **Kullanıcı - Hesap İlişkisi (One-to-Many/1:N):**

Bir müşteri, bankacılık hiyerarşisinde birden fazla hesaba (vadesiz, döviz vb.) sahip olabilir.

HESAP tablosu, kullanıcı_no (FK) üzerinden ana tabloya bağlıdır. İlişki çizgisi "Kullanıcı tarafında zorunlu 1, Hesap tarafında ise 0 veya N" şeklindedir; yani bir kullanıcı henüz hesabı olmasa bile sistemde var olabilir.

- **Hesap - İşlem İlişkisi (One-to-Many/1:N):**

Finansal hareketlerin takibi için her işlemin net bir kaynağı ve hedefi olmalıdır.

ISLEM tablosu, hem gonderen_hesap_no hem de alici_hesap_no üzerinden HESAP tablosuna iki farklı koldan bağlanır. Bu çift taraflı ilişki, bir hesabın geçmişte yaptığı tüm işlemleri "gönderen" veya "alan" sıfatıyla sorgulamamıza olanak tanır.

- **Kullanıcı - Giriş Takibi (One-to-Many/1:N):**

Sistemin dolandırıcılık tespiti yapabilmesi için kullanıcının geçmiş oturum bilgilerine ihtiyacı vardır.

Bir kullanıcı zaman içinde farklı cihazlardan ve konumlardan defalarca giriş yapabilir. GIRIS_TAKIP tablosundaki her kayıt, tek bir kullanıcı_no değerine işaret ederek o kullanıcının dijital ayak izini oluşturur.

- **İşlem - Risk Analizi (One-to-One/isteğe bağlı):**

Bu ilişki, sistemin en kritik kontrol noktasıdır. Her işlem doğası gereği şüpheli değildir; bu nedenle RISK_ANALIZI tablosu ile ISLEM tablosu arasında **isteğe bağlı (optional)** bir bağ vardır.

Sistem bir işlemi "Yüksek miktarlı" veya "Farklı konum" gibi kriterlerle riskli bulduğunda, bu işleme ait islem_no (FK) kullanılarak RISK_ANALIZI tablosunda tek bir kayıt oluşturulur. Böylece temiz işlemler için boş veri tutulmazken, şüpheli işlemler risk_seviyesi (low, medium, high) ile detaylandırılır.

3. VERİTABANI UYGULAMASI

3.1. Veritabanı Tablolarını Oluşturma (DDL)

Projenin temelini oluşturan veritabanı yapısı, ilişkisel veritabanı kurallarına uygun olarak normalize edilmiş ve veri bütünlüğünü koruyacak kısıtlamalarla donatılmıştır. Aşağıda kullanılan tablolar ve teknik detayları yer almaktadır.

```
-- Roller Tablosu
CREATE TABLE Roller (
    RolID INT PRIMARY KEY IDENTITY(1,1),
    RolAdi NVARCHAR(50) NOT NULL
);

-- Kullanıcılar Tablosu
CREATE TABLE Kullanıcılar (
    KullanıcıID INT PRIMARY KEY IDENTITY(1,1),
    AdSoyad NVARCHAR(100) NOT NULL,
    Eposta NVARCHAR(100) UNIQUE NOT NULL,
    Sifre NVARCHAR(255) NOT NULL,
    RolID INT,
    CONSTRAINT FK_Kullanıcı_Rol FOREIGN KEY (RolID)
    REFERENCES Roller(RolID)
);

-- Hesaplar Tablosu
CREATE TABLE Hesaplar (
    HesapID INT PRIMARY KEY IDENTITY(1,1),
    KullanıcıID INT,
    Bakiye DECIMAL(18,2) DEFAULT 0,
```

```

        HesapTuru NVARCHAR(30),
        OlusturulmaTarihi DATETIME DEFAULT GETDATE(),
        CONSTRAINT      FK_Hesap_Kullanici      FOREIGN      KEY
(KullaniciID)
        REFERENCES Kullanicilar(KullaniciID)
    );
-- Islemler Tablosu
CREATE TABLE Islemler (
    IslemID INT PRIMARY KEY IDENTITY(1,1),
    GonderenHesapID INT,
    AliciHesapID INT,
    Miktar DECIMAL(18,2) NOT NULL,
    IslemTarihi DATETIME DEFAULT GETDATE(),
    Konum NVARCHAR(100),
    CONSTRAINT FK_Gonderen FOREIGN KEY (GonderenHesapID)
REFERENCES Hesaplar(HesapID),
    CONSTRAINT FK_Alici FOREIGN KEY (AliciHesapID)
REFERENCES Hesaplar(HesapID)
);
-- Supheli Islem Uyarilari
CREATE TABLE DolandiricilikUyarilari (
    UyariID INT PRIMARY KEY IDENTITY(1,1),
    IslemID INT,
    RiskPuanı INT CHECK (RiskPuanı BETWEEN 0 AND 100),
    Durum NVARCHAR(30) DEFAULT 'Beklemede',
    TespitTarihi DATETIME DEFAULT GETDATE(),
    CONSTRAINT FK_Uyari_Islem FOREIGN KEY (IslemID)
REFERENCES Islemler(IslemID)
);
GO

```

Bu SQL kodunda, dolandırıcılık tespitine yönelik bir sistemin temel veritabanı yapısı oluşturulmuştur.

Roller tablosu ile sistemdeki kullanıcıların sahip olabileceği yetkiler tanımlanmıştır. Ardından gelen **Kullanıcılar** tablosu, kullanıcıya ait kimlik, iletişim ve giriş bilgilerini tutmakta olup her kullanıcı bir rol ile ilişkilendirilmiştir. **Hesaplar** tablosu, kullanıcılara ait finansal hesapları temsil eder ve her hesap belirli bir kullanıcıya bağlıdır; ayrıca hesap bakiyesi ve oluşturulma tarihi gibi bilgiler de burada saklanır. **İslemler** tablosu, hesaplar arasında gerçekleştirilen para transferlerini kaydeder ve gönderici ile alıcı hesaplar arasında ilişki kurularak işlem geçmişi takip edilebilir hale getirilmiştir. Son olarak **DolandiricilikUyarilari** tablosu, yapılan işlemler üzerinde gerçekleştirilen analizler sonucunda şüpheli görülen durumları kayıt altına almak için tasarlanmıştır. Bu tabloda her işlem için bir risk puanı atanmakta ve belirli bir aralıkta kontrol edilmektedir.

Genel olarak bu yapı, kullanıcı–hesap–işlem ilişkisini bütünlüklü şekilde ele alarak hem finansal hareketlerin izlenmesini hem de olası dolandırıcılık faaliyetlerinin tespit edilmesini sağlayacak şekilde kurgulanmıştır.

3.2. Trigger İşlemleri

```
CREATE TRIGGER trg_DolandiricilikKontrol
ON Islemler
AFTER INSERT
AS
BEGIN
    INSERT INTO DolandiricilikUyarilari
    (IslemID, RiskPuani, Durum)
SELECT
    IslemID,
    85,
    'Yuksek Risk'
FROM inserted
WHERE Miktar > 10000    END;    GO
```

Bu kodda, veritabanında belirli işlemleri otomatik olarak kontrol edebilmek ve manuel müdahaleye gerek kalmadan riskli durumları anında tespit edebilmek amacıyla bir **trigger** kullanılmıştır. Trigger'lar, tablo üzerinde gerçekleşen belirli olaylar sonrasında otomatik çalışan yapılardır ve bu sayede sistemin daha hızlı ve tutarlı çalışması sağlanır.

Bu kapsamda oluşturulan **trg_DolandiricilikKontrol** adlı tetikleyici, **Islemler** tablosuna yeni bir kayıt eklendikten sonra devreye girer. Eklenen kayıtlar **inserted** sanal tablosu üzerinden kontrol edilir ve işlem tutarı (**Miktar**) 10.000 TL'nin üzerindeyse bu işlem yüksek riskli olarak değerlendirilir. Bu durumda ilgili işlem için **DolandiricilikUyarilari** tablosuna otomatik olarak bir kayıt eklenir; işlem kimliği (**IslemID**), risk puanı (85) ve durum bilgisi ("Yuksek Risk") kaydedilir.

Bu yapı sayesinde sistem, şüpheli işlemleri anlık olarak tespit ederek dolandırıcılık analiz sürecini otomatik hale getirmektedir.

3.3. Stored Procedure – Riskli İşlemlerin Listelenmesi

Bu bölümde sistemde tespit edilen riskli işlemleri listelemek amacıyla bir **stored procedure** tanımlanmıştır.

sp_RiskliIslemleriListele prosedürü, dolandırıcılık uyarıları tablosunu temel olarak işlemler, hesaplar ve kullanıcılar tablolarını **JOIN** ile birleştirir ve gerekli bilgileri tek bir sorguda sunar. **SELECT** kısmında işlem tutarı, kullanıcı adı, risk puanı ve tarih gibi alanlar alınmaktadır.

JOIN kullanımı sayesinde tablolar arasındaki ilişkiler kurulmuş ve dağınık veriler anlamlı bir bütün haline getirilmiştir. Bu prosedür, karmaşık sorguların tekrar yazılmasını önleyerek riskli işlemlerin hızlı ve pratik bir şekilde görüntülenmesini sağlar.

```
CREATE PROCEDURE sp_RiskliIslemleriListele
AS
BEGIN

    SELECT
        D.UyariID,
        K.AdSoyad,
        I.Miktar,
        D.RiskPuani,
        D.Durum,
        D.TespitTarihi
    FROM DolandiricilikUyarilari D
    JOIN Islemler I
```

```

        ON D.IslemID = I.IslemID
    JOIN Hesaplar H
        ON I.GonderenHesapID = H.HesapID
    JOIN Kullanicilar K
        ON H.KullaniciID = K.KullaniciID

END;

GO

```

3.4. Function - Kullanıcının Toplam Gönderdiği Para

Bu bölümde, belirli bir kullanıcının toplam gönderdiği para miktarını hesaplamak amacıyla bir **function** tanımlanmıştır. **fn_ToplamIslemTutari** fonksiyonu, parametre olarak alınan **KullaniciID** değerine göre ilgili kullanıcının yaptığı işlemleri filtreler. Fonksiyon içinde tanımlanan **@Toplam** değişkeni, **SUM** fonksiyonu kullanılarak işlemlerin toplam tutarını hesaplamak için kullanılmıştır. **JOIN** yapısı ile işlemler ve hesaplar tabloları birbirine bağlanarak işlemlerin hangi kullanıcıya ait olduğu belirlenmiştir. Son olarak **ISNULL** ifadesi ile eğer kullanıcıya ait işlem yoksa NULL yerine 0 değeri döndürülmesi sağlanmıştır.

Bu yapı sayesinde, kullanıcı bazlı toplam işlem tutarı kolay ve tekrar kullanılabilir bir şekilde elde edilmektedir.

```

CREATE FUNCTION fn_ToplamIslemTutari
(
    @KullaniciID INT
)
RETURNS DECIMAL(18,2)
AS
BEGIN

    DECLARE @Toplam DECIMAL(18,2)

    SELECT

        @Toplam = SUM(I.Miktar)
    FROM Islemler I

```

```

JOIN Hesaplar H
      ON I.GonderenHesapID = H.HesapID
WHERE H.KullaniciID = @KullaniciID

RETURN ISNULL(@Toplam,0)

END;
GO

```

3.5. Örnek Veriler (INSERT)

Bu bölümde oluşturulan veritabanı yapısını test edebilmek amacıyla örnek veriler eklenmiştir. İlk olarak **Roller** tablosuna sistemde kullanılacak rol bilgileri girilmiştir. **Kullanıcılar** tablosuna örnek kullanıcılar eklenmiş ve bu kullanıcılar ilgili roller ile ilişkilendirilmiştir. **Hesaplar** tablosunda her kullanıcıya ait hesap bilgileri tanımlanmıştır. Son olarak **İşlemler** tablosuna hem normal hem de yüksek tutarlı bir işlem eklenerek sistemin davranışı test edilmiştir. Özellikle 10.000 TL üzerindeki işlem sayesinde trigger mekanizmasının çalışması ve otomatik olarak riskli işlem uyarısı oluşturması ve görülmesi amaçlanmıştır.

```

INSERT INTO Roller (RolAdi)
VALUES ('Admin'), ('Musteri');

INSERT INTO Kullanicilar
(AdSoyad,Eposta,Sifre,RolID)
VALUES
('Sude Demir','sudedemmir@mail.com','sifre123',2),
('Akin Onursoz','akon@mail.com','987654',2),
('Bayrak Yilmaz','bayyil@mail.com','bayY234',2);

INSERT INTO Hesaplar
(KullaniciID,Bakiye,HesapTuru)
VALUES
(1,15000,'Vadesiz'),
(2,5000,'Vadesiz'),

```

```
(3,20000,'Mevduat');
INSERT INTO Islemler
(GonderenHesapID,AliciHesapID,Miktar,Konum)
VALUES
(1,2,500,'Izmir'),          -- Normal işlem
(3,1,12500,'Ankara');      -- Riskli işlem (Trigger çalışır)
GO
```

3.6. Update Örneği

Veritabanında kayıtlı verilerin güncellenmesi, iş akışının takibi ve sistemin dinamik yapısının korunması açısından kritik bir öneme sahiptir. Bu bölümde **UPDATE** komutu, tespit edilen bir dolandırıcılık uyarısının analiz edilip edilmediğini belirtmek ve statik veriyi operasyonel bir sürece dönüştürmek amacıyla kullanılır. Teknik olarak, DolandiricilikUyarilari tablosundaki ilgili kaydın durumu **WHERE** koşuluyla sadece belirli bir ID hedef alınarak **"İncelendi"** şeklinde güncellenmiştir.

Bu hassas kısıtlama, tablodaki diğer verilerin yanlışlıkla değişmesini önleyerek veri bütünlüğünü en üst düzeyde korur. Böylece yöneticilerin, bekleyen ve çözülen riskli işlemleri anlık olarak raporlayabilmesi ve denetleyebilmesi sağlanır.

```
UPDATE DolandiricilikUyarilari
SET Durum = 'Incelendi'
WHERE UyarıID = 1;
```

3.7. Delete Örneği - Örnek İşlem Silme

Bu Bölümde veritabanı performansını ve veri güncelliğini korumak amacıyla kullanılan **DELETE** komutu, hatalı veya işlevini yitirmiş kayıtların sistemden temizlenmesini sağlar. Bu işlemde **WHERE** koşuluyla sadece belirli bir **IslemID** hedef alınarak, veri bütünlüğü korunmuş ve yanlışlıkla toplu veri silinmesi riski önlenmiştir.

```
DELETE FROM Islemler
WHERE IslemID = 1;
```

3.8. Gelişmiş Sorgular

Bu bölümde, sistemdeki verileri analiz etmek amacıyla farklı SQL sorguları oluşturulmuştur (Hangi kullanıcı ne kadar işlem yapmış, riskli işlemleri yapan kişiler, son 30 günde en çok işlem yapan kullanıcı vb.).

İlk sorguda kullanıcıların yaptıkları toplam işlem sayısı ve toplam işlem tutarı hesaplanmıştır. **JOIN** yapısı ile kullanıcılar, hesaplar ve işlemler tabloları birleştirilmiş, **GROUP BY** ile kullanıcı bazında gruptama yapılmış ve **HAVING** koşulu ile belirli bir tutarın üzerindeki kullanıcılar filtrelenmiştir.

İkinci sorguda dolandırıcılık uyarısı alan işlemler listelenerek hangi kullanıcıların riskli işlem yaptığı görüntülenmiştir. Bu sorguda uyarı, işlem, hesap ve kullanıcı tabloları **JOIN** ile birleştirilerek risk puanı ve durum bilgisi birlikte sunulmuştur.

Son sorguda ise son 30 gün içinde en çok işlem yapan kullanıcı tespit edilmiştir. **DATEADD** fonksiyonu ile tarih filtresi uygulanmış, işlemler kullanıcı bazında sayılarak **ORDER BY** ile en yüksek işlem sayısına sahip kullanıcı bulunmuştur.

Bu sorgular sayesinde sistem hem kullanıcı davranışlarını analiz edebilmekte hem de riskli ve yoğun işlem yapan kullanıcıları kolayca tespit edebilmektedir.

```
-- Hangi kullanıcı ne kadar işlem yapmış?
```

```
SELECT
    K.AdSoyad,
    COUNT(I.IslemID) AS ToplamIslemSayisi,
    SUM(I.Miktar) AS ToplamTutar
```

```
FROM Kullanicilar K
JOIN Hesaplar H
    ON K.KullaniciID = H.KullaniciID
JOIN Islemler I
    ON H.HesapID = I.GonderenHesapID
```

```
GROUP BY K.AdSoyad
HAVING SUM(I.Miktar) > 1000;
```

```
-- Riskli işlemleri yapan kişiler
```

```
SELECT
    K.AdSoyad,
```

```

        I.Miktar,
        D.RiskPuani,
        D.Durum

FROM DolandırıcılıkUyarıları D
JOIN İşlemler I
    ON D.IslemID = I.IslemID
JOIN Hesaplar H
    ON I.GonderenHesapID = H.HesapID
JOIN Kullanıcılar K
    ON H.KullanıcıID = K.KullanıcıID;

-- Son 30 günde en çok işlem yapan kullanıcı
SELECT TOP 1
    K.AdSoyad,
    COUNT(I.IslemID) AS IslemSayisi

FROM Kullanıcılar K
JOIN Hesaplar H
    ON K.KullanıcıID = H.KullanıcıID
JOIN İşlemler I
    ON H.HesapID = I.GonderenHesapID

WHERE I.IslemTarihi >= DATEADD(DAY,-30,GETDATE())

GROUP BY K.AdSoyad
ORDER BY IslemSayisi DESC;

```

3.9. Procedure Çalıştırma

Bu bölümde daha önce oluşturulan **sp_RiskliIslemleriListele** adlı stored procedure çalıştırılmıştır. **EXEC** komutu kullanılarak prosedür çağrılmış ve sistemde kayıtlı olan riskli işlemler liste halinde ekrana getirilmiştir.

Bu işlem sayesinde, dolandırıcılık uyarısı almış işlemler, kullanıcı bilgileri ve risk detayları tek bir sorgu yazmadan hızlı bir şekilde görüntülenebilmektedir.

Stored procedure kullanımı, hem kod tekrarını azaltmakta hem de sorguların daha düzenli ve yönetilebilir bir şekilde çalıştırılmasını sağlamaktadır.

```
EXEC sp_RiskliIslemleriListele;
```

3.10. Function Çağırma

Bu bölümde daha önce tanımlanan **fn_ToplamIslemTutari** fonksiyonu çalıştırılmıştır. **SELECT** ifadesi ile fonksiyon çağrılarak parametre olarak verilen **KullaniciID = 3** değerine sahip kullanıcının toplam gönderdiği işlem tutarı hesaplanmıştır.

Fonksiyon, ilgili kullanıcının hesapları üzerinden yaptığı tüm işlemleri toplayarak tek bir sayısal değer döndürmektedir. Bu kullanım sayesinde kullanıcı bazlı toplam işlem tutarı hızlı ve tekrar sorgu yazmaya gerek kalmadan elde edilebilmektedir.

```
SELECT dbo.fn_ToplamIslemTutari(3) AS ToplamIslemTutari;
```

3.11. Tüm Tabloları Gösterme

Bu bölümde veritabanında oluşturulan tüm tabloların içeriğini görüntülemek amacıyla **SELECT *** sorguları kullanılmıştır. Her tablo ayrı ayrı çağrılarak sistemdeki mevcut verilerin kontrol edilmesi sağlanmıştır.

Roller, Kullanıcılar, Hesaplar, İşlemler ve DolandırıcılıkUyarıları tabloları sırasıyla listelenerek veritabanının genel durumu ve tablolar arasındaki ilişkiler doğrulanmıştır.

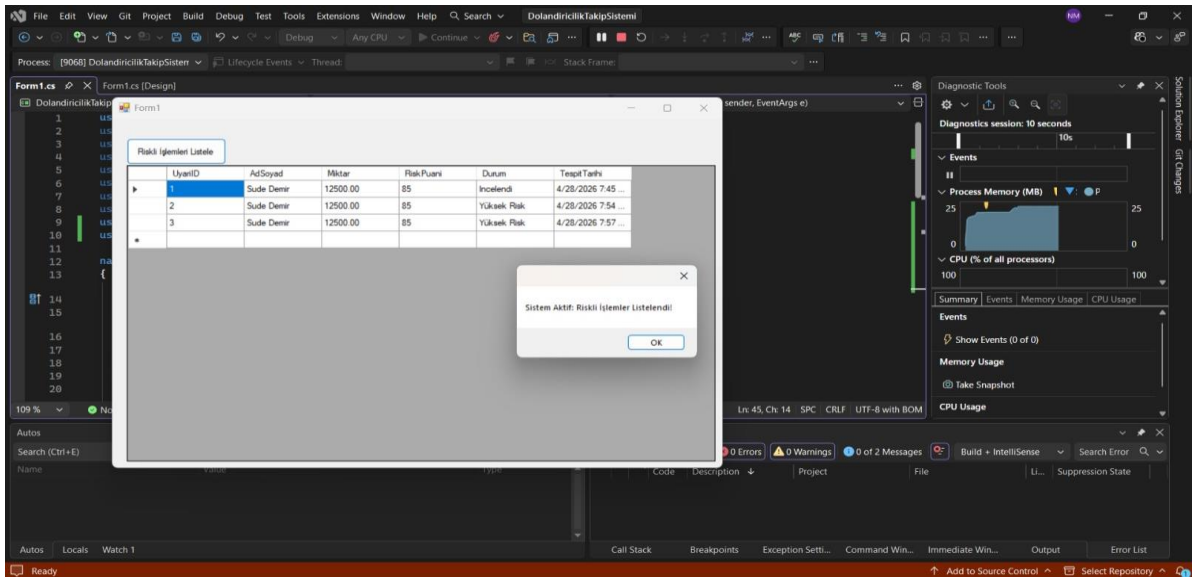
Bu işlem, hem verilerin doğru şekilde eklenip eklenmediğini kontrol etmek hem de sistemin bütünsel olarak çalışıp çalışmadığını test etmek için kullanılmaktadır.

```
SELECT * FROM Roller;  
SELECT * FROM Kullanıcılar;  
SELECT * FROM Hesaplar;  
SELECT * FROM İşlemler;  
SELECT * FROM DolandırıcılıkUyarıları;
```


4. UYGULAMA/ARAYÜZ ENTEGRASYONU

Projenin son aşamasında, veritabanında yapılandırılan mantıksal modelin kullanıcı düzeyinde test edilmesi ve görselleştirilmesi amacıyla bir C# Windows Forms arayüzü geliştirilmiştir. Bu arayüz, arka planda çalışan karmaşık SQL nesnelerini son kullanıcı için anlamlı bir operasyonel izleme paneline dönüştürmektedir.

Sistemin dinamik çalışma yapısını ve veritabanı bağlantısının başarısını kanıtlayan uygulama arayüzü sunulmuş olup bu tasarıma dair teknik detaylar aşağıda maddeler halinde açıklanmıştır.



Şekil 4.1. Uygulama arayüzü ve veritabanı bağlantı testi

4.1. C# Arayüz Entegrasyonu ve Veri Seçimi Mantığı (Teknik Özet)

Geliştirilen C# uygulamasında, veritabanındaki tüm veriler yerine özellikle "Riskli İşlemler" in listelenmesi tercih edilmiştir. Bu seçimin temel teknik gerekçeleri şunlardır:

Senaryo Odaklılık: Projenin temel amacı olan "Dolandırıcılık Tespit Sistemi" gereği, sistemin sıradan finansal hareketler arasından kritik olanları ayırt etme yeteneği ön plana çıkarılmıştır.

SQL Nesnelerinin Entegrasyonu: Bu ekran, veritabanı seviyesinde çalışan Trigger (trg_DolandiricilikKontrol) yapısının yakaladığı verileri, Stored Procedure (sp_RiskliIslemleriListele) aracılığıyla uygulama katmanına taşımaktadır. Bu sayede SQL nesnelerinin birbiriyle olan tam uyumu kanıtlanmıştır.

Operasyonel Verimlilik: Yönetici (Admin) ekranı tasarımı prensiplerine uygun olarak, kullanıcının binlerce kayıt arasında kaybolması engellenmiş; sadece müdahale gerektiren "Yüksek Riskli" kayıtlar raporlanarak sistemin çözüm odaklı yapısı vurgulanmıştır.

ADO.NET Bağlantısı: Uygulama, SqlConnection ve SqlDataAdapter sınıflarını kullanarak veritabanına dinamik olarak bağlanmakta ve veri bütünlüğünü koruyarak sonuçları sunmaktadır.

5. KAYNAKÇA

Bolton, C. (2010). *Pro SQL Server 2008 Relational Database Design and Implementation*. Apress.

Date, C. J. (2003). *An Introduction to Database Systems* (8. Baskı). Addison-Wesley.

Dokuz Eylül Üniversitesi. (2025-2026). *Bilgisayar Bilimleri Bölümü Ders Slaytları*. İzmir.

Elmasri, R., & Navathe, S. B. (2015). *Fundamentals of Database Systems* (7. Baskı). Pearson.

Google. (2026). *Gemini*. <https://gemini.google.com>

Microsoft. (2024). *ADO.NET Documentation*. <https://learn.microsoft.com/tr-tr/dotnet/framework/data/adonet/>

Microsoft. (2024). *SQL Server Technical Documentation*. <https://learn.microsoft.com/tr-tr/sql/sql-server/>

OpenAI. (2026). *ChatGPT*. <https://chat.openai.com>

Ramakrishnan, R., & Gehrke, J. (2003). *Database Management Systems* (3. Baskı). McGraw-Hill.

Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). *Database System Concepts* (7. Baskı). McGraw-Hill.

